

IMPACT ANALYSIS

CR030: Student Self-Registration with Automatic Student ID Generation

1. Change Summary

CR-030 introduces self-registration functionality for students in the Student Management System.

Students will be able to create their own accounts by completing a registration form without requiring the administrator to create the student account manually.

During registration, the system will automatically generate a unique Student ID using the existing database auto-increment mechanism. Student IDs must remain unique and must not be reused if a previously existing student record is deleted.

The change will also ensure that newly registered students are assigned the `STUDENT` role and are securely authenticated using the existing password hashing and login mechanisms.

2. Impact Assessment

Area	Impact	Assessment
Frontend	Moderate	A new student registration interface and validation logic will be added.
Backend	Moderate	A separate student self-registration process and controller logic will be required.
API	Low	A new public registration endpoint will be introduced for student registration.
Database	Moderate	Existing <code>AUTO_INCREMENT</code> , unique constraints, and student/user relationships can support the requirement.
Authentication	Moderate	Newly registered students must be able to log in using the existing authentication system.
Authorization	Low	Self-registration must create accounts only with the <code>STUDENT</code> role and must not provide administrator privileges.
Performance	Moderate	Registration involves normal database insert operations and does not introduce significant performance overhead.
CRUD	Low	Existing administrator Edit and Delete operations must continue to work correctly with automatically generated Student IDs.
Deployment	Moderate	Frontend and backend changes are required, but no major database migration is expected.
Student ID Generation	Low	The existing database <code>AUTO_INCREMENT</code> mechanism can generate unique Student IDs automatically.

3. Frontend Impact

The main frontend impact is the addition of a student self-registration interface.

Potential changes include:

- Student registration form.
- Registration number or required student details validation.
- First name and last name fields.
- Email and phone number fields.
- Password and confirmation validation where applicable.
- Registration success and error messages.
- Navigation from the login page to the registration page.
- Responsive registration interface.

The Student ID should not be manually entered or calculated by the student.

After successful registration, the system should receive the automatically generated Student ID from the backend and database.

The existing administrator student management interface should continue to display, edit, and delete students using their original `student_id`.

4. Backend, API and Database Impact

Backend changes are required to support student self-registration.

The existing administrator-only student creation process should remain protected for administrator use.

A separate student registration process should be introduced so that an unauthenticated user can register as a student without receiving administrator access.

The registration process should:

- Validate the submitted student information.
- Check for duplicate registration information and email addresses.
- Create the required user account.
- Assign the `STUDENT` role.
- Securely hash the student's password.
- Create the student record.
- Obtain the automatically generated Student ID from the database.
- Return the registration result to the frontend.

A new registration API endpoint may be introduced, for example:

```
POST /api/students/register
```

The existing database design already supports automatic Student ID generation through the `AUTO_INCREMENT` property of `students.student_id`.

The database also contains unique constraints for fields such as `registration_no` and `email`.

No manual Student ID calculation should be implemented using the number of existing students or `MAX(student_id) + 1`.

If a Student ID such as 5 is deleted, the next newly created student should receive a new ID rather than reusing 5.

5. Security Impact

Security impact is moderate because the change introduces a new public registration process.

The registration functionality must:

- Allow only student account creation.
- Automatically assign the `STUDENT` role.
- Prevent users from selecting or changing their role during registration.
- Never allow self-registration to create an administrator account.
- Validate all required input fields.
- Prevent duplicate registration numbers.
- Prevent duplicate email addresses.
- Hash passwords before storing them in the database.
- Avoid storing plain-text passwords.
- Maintain the existing JWT authentication mechanism after registration.
- Preserve existing administrator authorization controls.
- Prevent unauthorized users from accessing administrator functions.

The existing authentication and authorization mechanisms should remain unchanged for existing users.

6. CRUD and Functional Impact

The change affects student creation because students will be able to create their own accounts.

Existing administrator CRUD operations must continue to work correctly after self-registration is introduced.

Particular attention is required for:

- Creating a student through self-registration.
- Automatically generating the Student ID.
- Maintaining unique Student IDs.
- Preventing deleted Student IDs from being reused.
- Editing newly registered students.
- Deleting newly registered students.
- Maintaining the correct `student_id` during Edit and Delete operations.
- Maintaining the relationship between the `users` and `students` records.

Student ID generation must be controlled by the database rather than by frontend logic.

For example:

Existing students:

```
Student ID 1
Student ID 2
Student ID 3
Student ID 4
```

If Student ID 3 is deleted:

```
Student ID 1
Student ID 2
Student ID 4
```

When another student registers, the new Student ID should be:

```
Student ID 5
```

The deleted ID 3 must not be reused.

7. Performance Impact

The expected performance impact is low.

Student registration requires database operations for creating the user and student records. These operations are performed only when a student submits the registration form.

The database `AUTO_INCREMENT` mechanism provides efficient ID generation and avoids the need to retrieve the entire student list or calculate the next ID manually.

No continuous API requests are required during registration.

The current approach should be suitable for the expected number of student registrations.

8. Testing Impact

- Manual and functional testing should cover:
- Opening the student registration page.
- Successful student registration.
- Required field validation.
- Invalid email validation.
- Duplicate registration number validation.
- Duplicate email validation.
- Password validation.
- Successful password hashing.
- Automatic Student ID generation.
- Sequential Student ID generation.
- Student ID uniqueness.
- Deleting an existing student.
- Registering another student after deletion.
- Confirming that the deleted Student ID is not reused.
- Confirming that a new Student ID is generated.
- Student login after successful registration.
- Correct assignment of the `STUDENT` role.
- Preventing self-registration as an administrator.
- Existing administrator login.
- Existing administrator Edit functionality.
- Existing administrator Delete functionality.
- Existing student functionality.
- Database transaction and error handling.

9. Risk Analysis

Risk	Impact	Mitigation
Duplicate Student ID	High	Use the database <code>AUTO_INCREMENT</code> mechanism instead of manual ID calculation.
Deleted Student ID is reused	High	Ensure Student IDs are generated by the database and never manually reset or calculated.
Duplicate registration number	High	Maintain the existing unique database constraint and validate before insertion.
Duplicate email address	High	Validate email uniqueness before creating the student account.
Student creates an administrator account	Critical	Always assign the <code>STUDENT</code> role on self-registration and do not accept a role from the client.
Plain-text password storage	Critical	Hash passwords using the existing password hashing mechanism before database insertion.
Existing admin CRUD functionality breaks	High	Perform regression testing for Edit and Delete operations.
Registration creates incomplete records	High	Use database transactions so user and student records are created consistently.
Invalid registration data	Medium	Apply frontend and backend validation.
Unauthorized access to admin functions	High	Preserve existing authentication and role-based authorization middleware.
Registration process fails due to database error	Medium	Implement transaction rollback and suitable error handling.
Multiple users register simultaneously	Medium	Rely on database-generated IDs and unique constraints to prevent duplicate IDs and records.

10. Overall Impact and Recommendation

Overall Impact: Low

CR-030 is a functional enhancement that affects the frontend, backend, API, authentication, authorization, and student creation process.

The database already provides important support for this requirement through the `AUTO_INCREMENT` Student ID and unique constraints. Therefore, a major database redesign is not expected.

The change is technically feasible and recommended for implementation, subject to formal approval and testing.

Key implementation considerations:

- Student ID must be generated automatically by the database.
- Student IDs must remain unique.
- Deleted Student IDs must never be reused.
- Students must not manually enter or calculate their Student ID.
- Self-registration must always create a `STUDENT` account.
- Administrator privileges must not be available through self-registration.
- Existing administrator CRUD operations must continue to work correctly.
- User and student records should be created within a transaction to maintain data consistency.